Spacewise | Technical Report

Hack Apertus 2026 | Track 2B - Own Project | Individual participant: Avishek Nath

Repository: https://github.com/newstartech1/spacewise/tree/hack-apertus

1. Problem and objective

Spacewise is a room-understanding and spatial-improvement prototype intended to help people make more practical use of existing indoor spaces. Users supply a photograph of a room; the system extracts visual signals and prepares structured recommendations rather than requiring the user to manually inventory every visible object.

2. Implemented architecture

Web client: HTML, CSS, and JavaScript frontend for the room-analysis workflow. **API:** Python FastAPI backend for receiving images and returning structured results. **Image services:** AWS S3 for image storage and Amazon Rekognition for image-label detection. **Assessment:** a Python assessment module for evaluating recommendations. **Language-model adapter:** OpenAI-compatible HTTP chat-completion request structure configurable through environment variables.

3. Analysis workflow

The intended processing sequence is: (1) accept a room image; (2) store/process the image through the configured AWS services; (3) extract visible inventory and image properties from detection labels; (4) pass structured context to the language-model recommendation component; (5) return detected facts, recommendations, and limitations in a consistent API response. The code includes deterministic fallback logic when model credentials are not configured.

4. Apertus integration and exact verification status

The hackathon branch adds a configurable model-inference adapter using **APERTUS_API_KEY**, **APERTUS_BASE_URL**, and **APERTUS_MODEL**. During development, the Hugging Face router was configured for **swiss-ai/Apertus-8B-Instruct-2509** (the earlier 8B release). A live API request returned HTTP **402 Payment Required** because inference credits were exhausted. The organizers subsequently announced release of additional credits, but successful live inference and an upgrade to **Apertus 8B v1.5** had **not been verified** when this report was prepared. Thus, this report does not claim a completed or validated v1.5 deployment.

5. Testing and limitations

The repository contains backend/test_assessment.py, but this report does not claim that the entire end-to-end application or model integration has passed tests. Current limitations include dependence on AWS service configuration, Hugging Face inference availability and credits, model-version compatibility, and the absence of a verified live demo at submission preparation time. Image labels alone may omit spatial relationships, measurements, or objects; recommendations should therefore be treated as suggestions, not architectural or safety certifications.

6. Next engineering steps

Once inference access is available: (1) identify an accessible official Apertus 8B v1.5 endpoint; (2) update the model identifier and verify a real response; (3) exercise the full image-to-recommendation path; (4) compare recommendations against visible room facts and limitations; (5) record a reproducible demonstration and document setup and test outcomes.

7. Source and reproducibility

The **hack-apertus** branch contains the project changes for this submission. Runtime API keys are supplied via environment variables and should not be committed to the repository. The repository is the authoritative reference for implementation details and current project state.